NL-to-SQL Lakehouse

Research-format document on semantic-layered natural-language analytics over a lakehouse.

Sai Veda

Independent Project Research

2024

Contents

1. Executive Summary

This brief captures a natural-language analytics system built on a real lakehouse pattern: a large partitioned warehouse, a semantic layer, and a deterministic text-to-SQL compiler with safety checks. The project demonstrates that natural-language analytics becomes much more credible when it is backed by a semantic layer and SQL validation. The result is a system that answers business questions in English while still producing auditable SQL. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: DuckDB star schema + semantic layer + text-to-SQL, implementation evidence, benchmark results, project validation

2. Analytics Context

Business users ask in plain language, but trusted reporting still depends on correct joins, guarded metrics, and a stable schema contract. The goal was to bridge those worlds without hiding behind an opaque model call. A DuckDB lakehouse over a partitioned-Parquet star schema, with a semantic layer and a deterministic, offline text-to-SQL engine. Ask a business question in English - "top 5 product categories by revenue", "which region has the lowest revenue?", "monthly revenue in 2023" - and the system links schema entities, compiles validated SQL through the semantic layer, and executes it out-of-core against 50M fact rows. No paid LLM, no API keys, fully seeded and reproducible. The NL engine sits behind an interface so a real LLM can drop in without touching the validation or execution path.

2.1 Trusted NL-to-SQL Problem Definition

The central project problem can be stated directly. Business users ask in plain language, but trusted reporting still depends on correct joins, guarded metrics, and a stable schema contract. The goal was to bridge those worlds without hiding behind an opaque model call. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Review Questions

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: 50M-row lakehouse; 100% NL->SQL execution accuracy.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 36 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Semantic And Warehouse Foundations

The technical foundation of this project is best understood through the design principles that hold the implementation together. The semantic layer and SQL guardrails are the load-bearing parts that keep natural-language analytics trustworthy.

3.1 Semantic And Warehouse Foundations

1. The semantic layer is what makes NL-to-SQL trustworthy rather than merely executable. The semantic layer and SQL guardrails are the load-bearing parts that keep natural-language analytics trustworthy.
2. DuckDB was chosen because it keeps the warehouse story local, fast, and transparent.
3. Validation is a product feature: unsafe or schema-breaking queries should fail early and clearly.

3.2 Repository Examples And Signals

Keeping metric semantics in one place means "revenue" is defined once and is identical whether asked by the NL engine, a notebook, or a future LLM. This is the load-bearing safety story for text-to-SQL: swap the deterministic generator for an LLM and the guardrail is unchanged. Takeaway: on lakehouse layouts, row-group and file sizing is a first-order performance lever. Partition for pruning, but coalesce to few, large files.

- Fact and dimension generation into partitioned Parquet.
- DuckDB execution layer over the lakehouse.
- Entity linking, metric mapping, and validated SQL generation.

4. Lakehouse Architecture And Compiler Path

The semantic layer and SQL guardrails are the load-bearing parts that keep natural-language analytics trustworthy. This is the load-bearing safety story for text-to-SQL: swap the deterministic generator for an LLM and the guardrail is unchanged. Takeaway: on lakehouse layouts, row-group and file sizing is a first-order performance lever. Partition for pruning, but coalesce to few, large files.

4.1 Implementation Method

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Warehouse design: Modeled the dataset as a star schema so business questions map cleanly to measures and dimensions. Semantic contract: Added a metric layer to keep NL questions aligned with curated business logic. Compiler path: Converted linked entities and intents into validated SQL that can be inspected before execution. Performance review: Measured heavy queries at 50M rows to show that the analytic path remains credible at scale.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Evaluation Setup And Data Evidence

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 36 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 0 benchmark table(s), 4 screenshot asset(s), and 36 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Stand up a large star-schema lakehouse that remains queryable out of core. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes benchmarks/accuracy_by_category.csv, benchmarks/generation_report.json, benchmarks/scaling_results.csv, benchmarks/suite_results.csv, benchmarks/suite_summary.json.

6. Analysis Findings

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: 50M-row lakehouse; 100% NL->SQL execution accuracy. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The project demonstrates that natural-language analytics becomes much more credible when it is backed by a semantic layer and SQL validation. The result is a system that answers business questions in English while still producing auditable SQL. Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone. The lakehouse benchmarks show that large-table scans and joins remain practical on one machine.

6.2 Interpretation

Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone. The lakehouse benchmarks show that large-table scans and joins remain practical on one machine. The screenshots reinforce that trust comes from visible SQL and visible schema linkage, not from hidden prompt logic.

The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Analytic Figures

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

7.1 Star schema (data model)

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone.

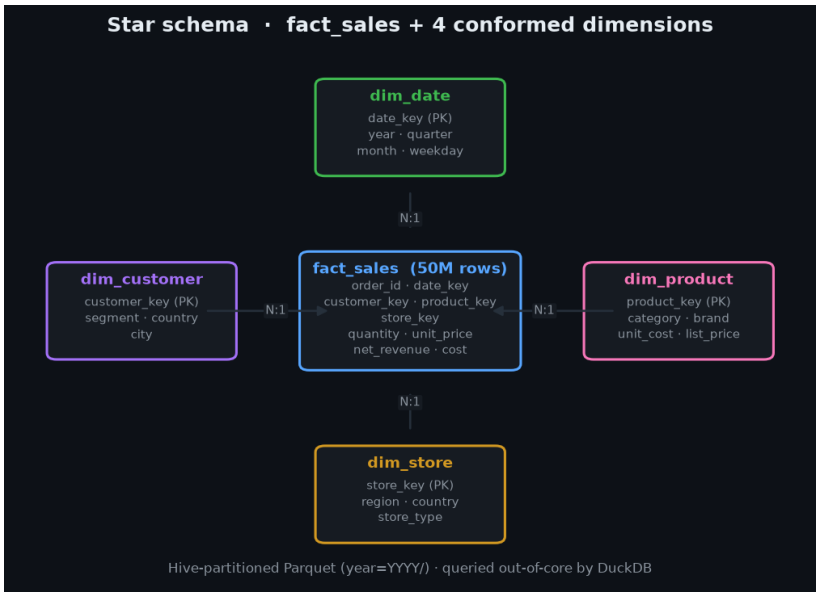

Figure 7.1. Star schema (data model)

7.2 NL->SQL execution accuracy by category

The visual evidence attached to NL->SQL execution accuracy by category helps connect the quantitative story to the implemented workflow. The lakehouse benchmarks show that large-table scans and joins remain practical on one machine.

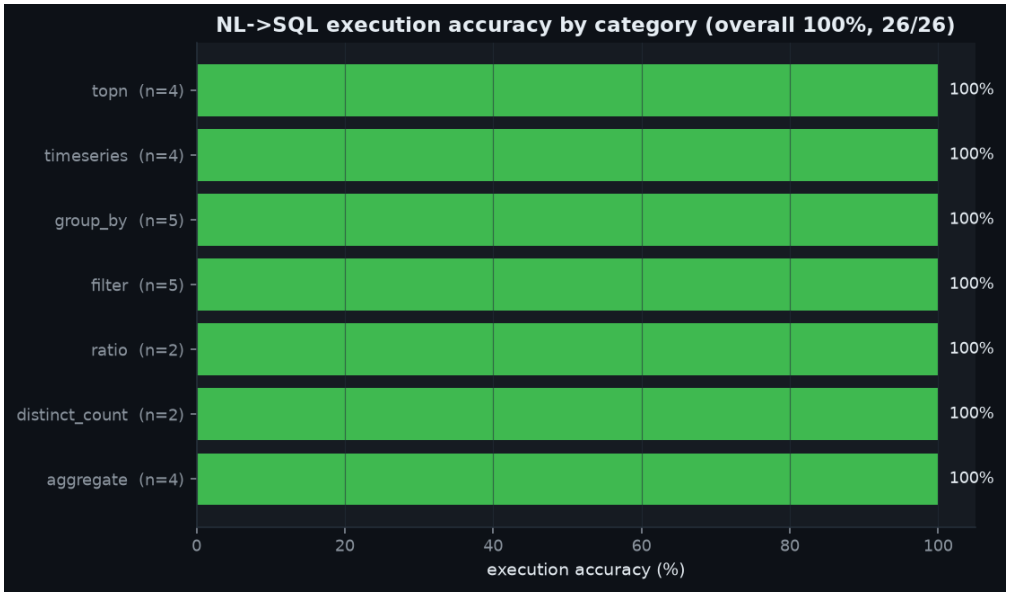

Figure 7.2. NL->SQL execution accuracy by category

8. Validation Notes And Evidence Check

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

8.1 Query latency vs data scale (log-log)

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. The screenshots reinforce that trust comes from visible SQL and visible schema linkage, not from hidden prompt logic.

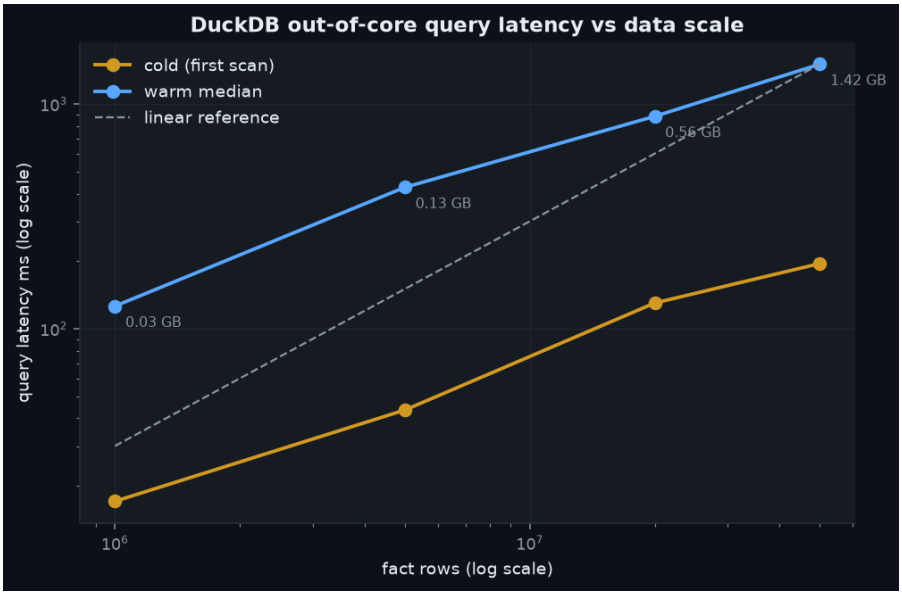

Figure 8.1. Query latency vs data scale (log-log)

8.2 Analytics answered from natural language

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Handled through explicit entity linking and a semantic metric layer.

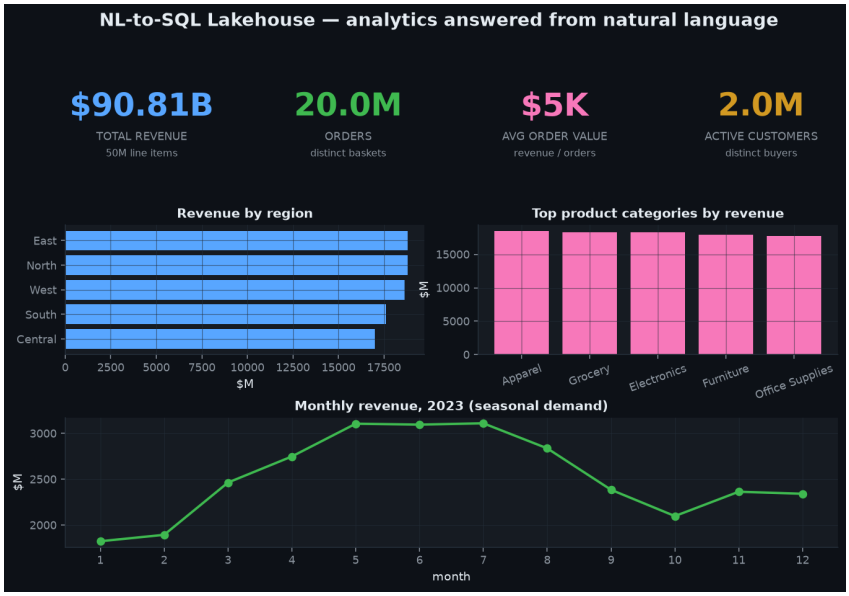

Figure 8.2. Analytics answered from natural language

8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	36 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and he
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Trust And Usefulness Discussion

The project demonstrates that natural-language analytics becomes much more credible when it is backed by a semantic layer and SQL validation. The result is a system that answers business questions in English while still producing auditable SQL. Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone. The lakehouse benchmarks show that large-table scans and joins remain practical on one machine. The screenshots reinforce that trust comes from visible SQL and visible schema linkage, not from hidden prompt logic. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Current Constraints

Schema ambiguity: Handled through explicit entity linking and a semantic metric layer. Unsafe SQL: Managed with validation gates before query execution is allowed. Metric drift: Reduced by centralizing business definitions instead of encoding them ad hoc in prompts.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Document Conclusion

This project document concludes that NL-to-SQL Lakehouse is best understood as a complete technical implementation supported by repository-native evidence. This brief captures a natural-language analytics system built on a real lakehouse pattern: a large partitioned warehouse, a semantic layer, and a deterministic text-to-SQL compiler with safety checks.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Planned Expansion

- Support more complex cohort, funnel, and time-series comparison queries.
- Add approval workflows for generated SQL in regulated or finance-facing contexts.
- Connect the semantic layer to upstream catalog metadata and freshness indicators.

11. Evidence Appendix

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/accuracy_by_category.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/generation_report.json	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/scaling_results.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/suite_results.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/star_schema.png	Screenshot	Visual validation surface referenced as 'Star schema (data model)' in the document review.
assets/accuracy_by_category.png	Screenshot	Visual validation surface referenced as 'NL->SQL execution accuracy by category' in the document review.
assets/latency_vs_scale.png	Screenshot	Visual validation surface referenced as 'Query latency vs data scale (log-log)' in the document review.
assets/analytics_dashboard.png	Screenshot	Visual validation surface referenced as 'Analytics answered from natural language' in the document review.
tests/	Validation	Automated test suite containing 36 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.

3. benchmarks/accuracy_by_category.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/generation_report.json - benchmark or result artefact used in the analysis section.
5. benchmarks/scaling_results.csv - benchmark or result artefact used in the analysis section.
6. benchmarks/suite_results.csv - benchmark or result artefact used in the analysis section.
7. tests/ - automated validation suite used to support implementation claims.
8. assets/accuracy_by_category.png - generated screenshot evidence included in the report.